

# 廈門大學



## 信息学院软件工程系

### 《计算机网络》实验报告

题    目 实验五 利用 Socket API 实现许可认证软件

班    级 软件工程 2023 级数媒班

姓    名 马鑫

学    号 37220232203780

实验时间 2025 年 11 月 21 日

2025 年 2 月 15 日

# 填写说明

- 1、本文件为 Word 模板文件，建议使用 Microsoft Word 2024 打开，在可填写的区域中如实填写；
- 2、填表时勿改变字体字号，保持排版工整，打印为 PDF 文件提交；
- 3、文件总大小尽量控制在 1MB 以下，最大勿超过 5MB；
- 4、在实验课结束 14 天内，按实验报告提交到我校课程网站的指定位置，源代码等主要材料上传在公开的代码托管平台上。
- 5、鼓励同学之间探讨，鼓励合理使用人工智能平台，提升效率，但不应滥用相关资源，如抄袭代码和代写作业。

## 1 实验目的

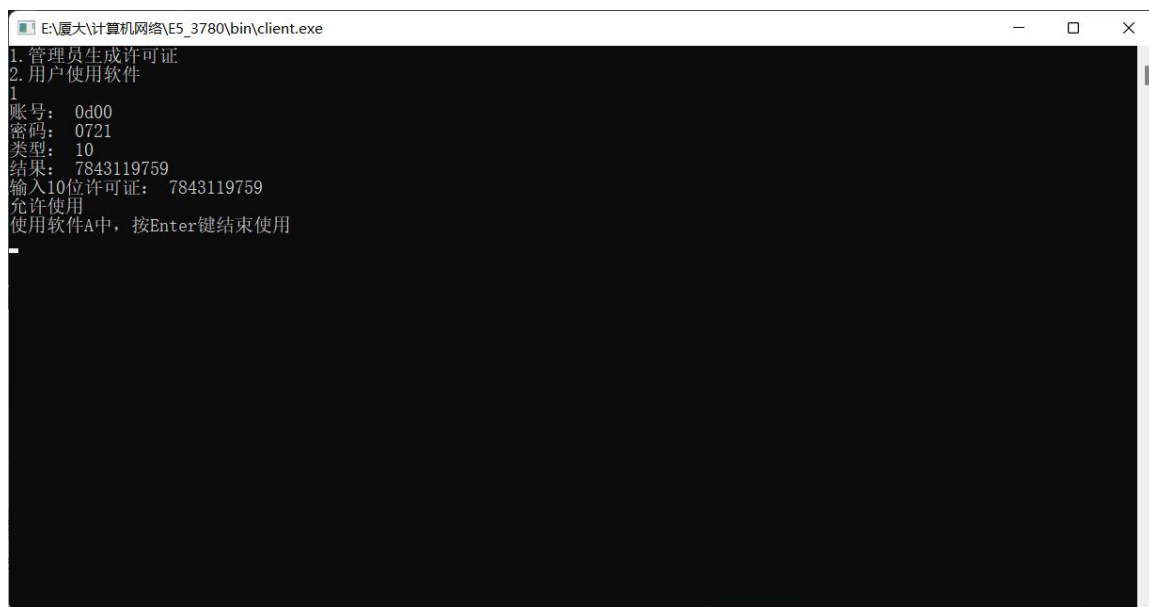
通过完成实验，掌握应用层文件传输的原理；了解传输过程中传输层协议选用、应用层协议设计和协议开发等概念。

## 2 实验环境

vs2022,phpstudy8.1.1.3

## 3 实验结果

（截图，每幅图配以简要的文字说明。勿长篇大论，贴核心代码，勿贴非核心代码。本段话删除。）



客户端端口，输入 1 可创建许可证，并输入序列号使用软件 A。

```
E:\code-vs\CNhomework\x64\Debug\CNhomework.exe
2025-11-28 20:09:38 许可证服务器启动 (端口8888)
2025-11-28 20:10:01 用户: 0d00接入, 并分配许可证7843119759
2025-11-28 20:10:14 客户接入, 许可证: 7843119759, IP地址: 127.0.0.1, 用户数量: 1
2025-11-28 20:10:14 接收到许可证为7843119759, ip地址为127.0.0.1的用户状态报送
2025-11-28 20:10:20 客户接入, 许可证: 7843119759, IP地址: 127.0.0.1, 用户数量: 2
2025-11-28 20:10:20 接收到许可证为7843119759, ip地址为127.0.0.1的用户状态报送
2025-11-28 20:10:25 客户接入, 许可证: 7843119759, IP地址: 127.0.0.1, 用户数量: 3
2025-11-28 20:10:25 接收到许可证为7843119759, ip地址为127.0.0.1的用户状态报送
2025-11-28 20:10:30 客户接入, 许可证: 7843119759, IP地址: 127.0.0.1, 用户数量: 4
2025-11-28 20:10:30 接收到许可证为7843119759, ip地址为127.0.0.1的用户状态报送
2025-11-28 20:10:36 客户接入, 许可证: 7843119759, IP地址: 127.0.0.1, 用户数量: 5
2025-11-28 20:10:36 接收到许可证为7843119759, ip地址为127.0.0.1的用户状态报送
2025-11-28 20:10:41 许可证为7843119759, ip地址为127.0.0.1的客户接入失败
2025-11-28 20:10:41 客户端127.0.0.1断开连接
2025-11-28 20:10:57 许可证为7843119759, ip为127.0.0.1的用户退出
2025-11-28 20:10:58 许可证为7843119759, ip为127.0.0.1的用户退出
2025-11-28 20:10:59 许可证为7843119759, ip为127.0.0.1的用户退出
2025-11-28 20:10:59 许可证为7843119759, ip为127.0.0.1的用户退出
```

服务器端口, 监听许可证和用户的状态, 每个许可证允许接入 5 个用户, 如果超过 5 个用户, 则拒绝接入。



| 许可证序列号     | 当前在线用户数 | 剩余许可数 | 许可证状态 |
|------------|---------|-------|-------|
| 7843119759 | 1       | 4     | 可用    |

php 网页通过表格展示许可证数据。

```

void licenseListener(SOCKET licenseSocket)
{
    SOCKET clientSocket;
    struct sockaddr_in clientAddress;
    int clientAddressLength = sizeof(clientAddress);
    while (true)
    {
        clientSocket = accept(licenseSocket, (struct sockaddr*)&clientAddress, &clientAddressLength);
        if (clientSocket == INVALID_SOCKET)
        {
            std::cout << "接受许可证连接失败" << std::endl;
            continue;
        }
        char account[80], password[80], type[80];
        if (recv(clientSocket, account, 1024, 0) < 0)
        {
            std::cerr << "接受数据失败" << std::endl;
            closesocket(licenseSocket);
            continue;
        }
        if (recv(clientSocket, password, 1024, 0) < 0)
        {
            std::cerr << "接受数据失败" << std::endl;
            closesocket(licenseSocket);
            continue;
        }
        if (recv(clientSocket, type, 1024, 0) < 0)
        {
            std::cerr << "接受数据失败" << std::endl;
            closesocket(licenseSocket);
            continue;
        }
        std::string license = generateSerial();
        license_map[license] = 0;
        saveLicenseData();
    }
}

```

许可证监听线程，通过 Socket API 获取客户端用户的输入，并分配给其一个随机的序列号。

```

void clientThread(SOCKET client_socket)
{
    char buffer[1024] = { 0 };
    struct sockaddr_in client_addr;
    int addr_len = sizeof(client_addr);
    getpeername(client_socket, (struct sockaddr*)&client_addr, &addr_len);
    char client_ip[INET_ADDRSTRLEN];
    inet_ntop(AF_INET, &client_addr.sin_addr, client_ip, INET_ADDRSTRLEN);

    int bytesRead = recv(client_socket, buffer, 1024, 0);
    if (bytesRead < 0)
    {
        std::cerr << time_now() << "接受来自" << client_ip << "的消息失败" << std::endl;
        return;
    }
    std::string command(buffer);
    if (license_map.find(command) == license_map.end() || license_map[command] < max_license)
    {
        std::string res = "ACCEPT";
        send(client_socket, res.c_str(), res.length() + 1, 0);
        license_map[command]++;
        saveLicenseData();
        std::cout << time_now() << " 客户接入, " << "许可证: " << command << ", IP地址: " << client_ip << ", 用户数量: " << license_map[command] << std::endl;
        while (true)
        {
            bytesRead = recv(client_socket, buffer, 1024, 0);
            if (bytesRead > 0)
            {
                std::cout << time_now() << " 接收到许可证为 " << command << ", ip地址为 " << client_ip << "的用户状态报送" << std::endl;
            }
            else
            {
                std::cout << time_now() << " 许可证为 " << command << ", ip为 " << client_ip << "的用户退出" << std::endl;
                license_map[command]--;
                saveLicenseData();
                return;
            }
        }
    }
}

```

```

void serverListener(SOCKET serverSocket)
{
    SOCKET clientSocket;
    struct sockaddr_in clientAddress;
    int clientAddressLength = sizeof(clientAddress);
    while (true)
    {
        clientSocket = accept(serverSocket, (struct sockaddr*)&clientAddress, &clientAddressLength);
        if (clientSocket == INVALID_SOCKET)
        {
            std::cout << "接受用户连接失败" << std::endl;
            continue;
        }
        std::thread(clientThread, clientSocket).detach();
    }
}

```

用户监听线程，通过并发实现允许多个用户接入，并获取用户的 ip 地址。

```

std::thread licenseThread(licenseListener, licenseSocket);
std::thread serverThread(serverListener, serverSocket);
licenseThread.join();
serverThread.join();

```

许可证和用户线程并发实现服务端架构。

```

//手动构造JSON
void saveLicenseData()
{
    std::ofstream file("license_data.json");
    if (!file.is_open())
    {
        std::cerr << time_now() << " 无法写入" << std::endl;
        return;
    }
    file << "{\n\t\"max_license\": \" " << max_license << ",\n\t\"server_start_time\": \" " << server_start_time << "\",\n\t\"updated_time\n";
    bool first = true;
    for (auto& pair : license_map)
    {
        if (!first)
        {
            file << ",\n";
        }
        first = false;
        file << "\t\t{\n\t\t\t\"serial\": \" " << pair.first << "\",\n\t\t\t\"user_count\": \" " << pair.second << "\"\n\t\t}";
    }
    file << "\n\t]\n";
    file.close();
}

```

通过手动构造 json 文件用于 php 网页端读取许可证信息。



```

int main()
{
    std::cout << "1.管理员生成许可证" << std::endl;
    std::cout << "2.用户使用软件" << std::endl;
    int choice;
    std::cin >> choice;
    if (choice == 1)
    {
        std::string account, password, type;
        std::cout << "账号: ";
        std::cin >> account;
        std::cout << "密码: ";
        std::cin >> password;
        std::cout << "类型: ";
        std::cin >> type;
        std::string serial = get_license(account.c_str(), password.c_str(), type.c_str());
        std::cout << "结果: " << serial << std::endl;
    }
}
WSADATA wsaData;

```

```

std::string serial;
std::cout << "输入10位许可证: ";
while (std::cin >> serial && serial.size() != 10)
{
    std::cout << "许可证必须为10位";
}

if (connect(clientSocket, (struct sockaddr*)&serverAddress, sizeof(serverAddress)) < 0)
{
    std::cerr << "连接服务器失败" << std::endl;
    return -1;
}

if (send(clientSocket, serial.c_str(), serial.size() + 1, 0) < 0)
{
    std::cerr << "发送消息失败" << std::endl;
    return -1;
}

char buffer[1024] = { 0 };
if (recv(clientSocket, buffer, 1024, 0) < 0)
{
    std::cerr << "接受响应失败" << std::endl;
    return -1;
}

std::string s = buffer;
if (s == "ACCEPT")
{
    std::cout << "允许使用" << std::endl;
    isStop = false;
    std::thread t(sendStatusReport, clientSocket);
    softwareA();
    isStop = true;
    t.detach();
}
else if (s == "REJECT")
{
    std::cout << "当前许可证已满" << std::endl;
    Sleep(10);
}

closesocket(clientSocket);

```

客户端输入 1 可输入管理员信息，否则输入序列号使用软件 A。

```
void sendStatusReport(SOCKET client_socket)
{
    while (!isStop)
    {
        std::string report = "status report";
        if (send(client_socket, report.c_str(), report.size() + 1, 0) < 0)
        {
            std::cerr << "状态发送失败" << std::endl;
        }
        std::this_thread::sleep_for(std::chrono::seconds(TIMEOUT));
    }
}

std::thread t(sendStatusReport, clientSocket);
```

通过并发开启发送状态的线程，每 30min 发送一次状态信息。

```
// 读取JSON文件
if (file_exists($jsonFile)) {
    $jsonContent = file_get_contents($jsonFile);
    $data = json_decode($jsonContent, true);
}
?>

<?php if (!empty($data)): ?>
    <div class="status-item">服务器启动时间: <?php echo htmlspecialchars($data['server_start_time']); ?></div>
    <div class="status-item">数据最后更新: <?php echo htmlspecialchars($data['update_time']); ?></div>
    <div class="status-item">单许可证最大用户数: <?php echo $data['max_license']; ?></div>
<?php else: ?>
    <div class="status-item">服务器未运行或数据文件不存在</div>
<?php endif; ?>
</div>
```



```
<!-- 许可证列表 -->
<table>
  <thead>
    <tr>
      <th>许可证序列号</th>
      <th>当前在线用户数</th>
      <th>剩余许可数</th>
      <th>许可证状态</th>
    </tr>
  </thead>
  <tbody>
    <?php if (!empty($data['licenses']) && count($data['licenses']) > 0): ?>
      <?php foreach ($data['licenses'] as $license): ?>
        <tr>
          <td><?php echo htmlspecialchars($license['serial']); ?></td>
          <td><?php echo $license['user_count']; ?></td>
          <td><?php echo $data['max_license'] - $license['user_count']; ?></td>
          <td>
            <?php if ($license['user_count'] >= $data['max_license']): ?>
              <span class="status-full">已满</span>
            <?php else: ?>
              <span class="status-available">可用</span>
            <?php endif; ?>
          </td>
        </tr>
      <?php endforeach; ?>
    <?php else: ?>
      <tr>
        <td colspan="4" class="empty-data">暂无许可证数据</td>
      </tr>
    <?php endif; ?>
  </tbody>
</table>
```

php 网页端读取 json 文件的数据并通过表格呈现数据。

## 4 实验代码

本次实验的代码已上传于以下代码仓库：  
[https://gitee.com/koshistar/cni-exp/tree/master/E5\\_3780](https://gitee.com/koshistar/cni-exp/tree/master/E5_3780)。

## 5 课后思考题

无

## 6 实验总结

通过本实验对 Socket 编程和应用层有了初步的了解，并通过线程并发实现多种功能，对操作系统也更加熟悉，首次编写了 php 代码，使用了 phpstudy 软件实现了网页，对动态网页有了初步的认知。